

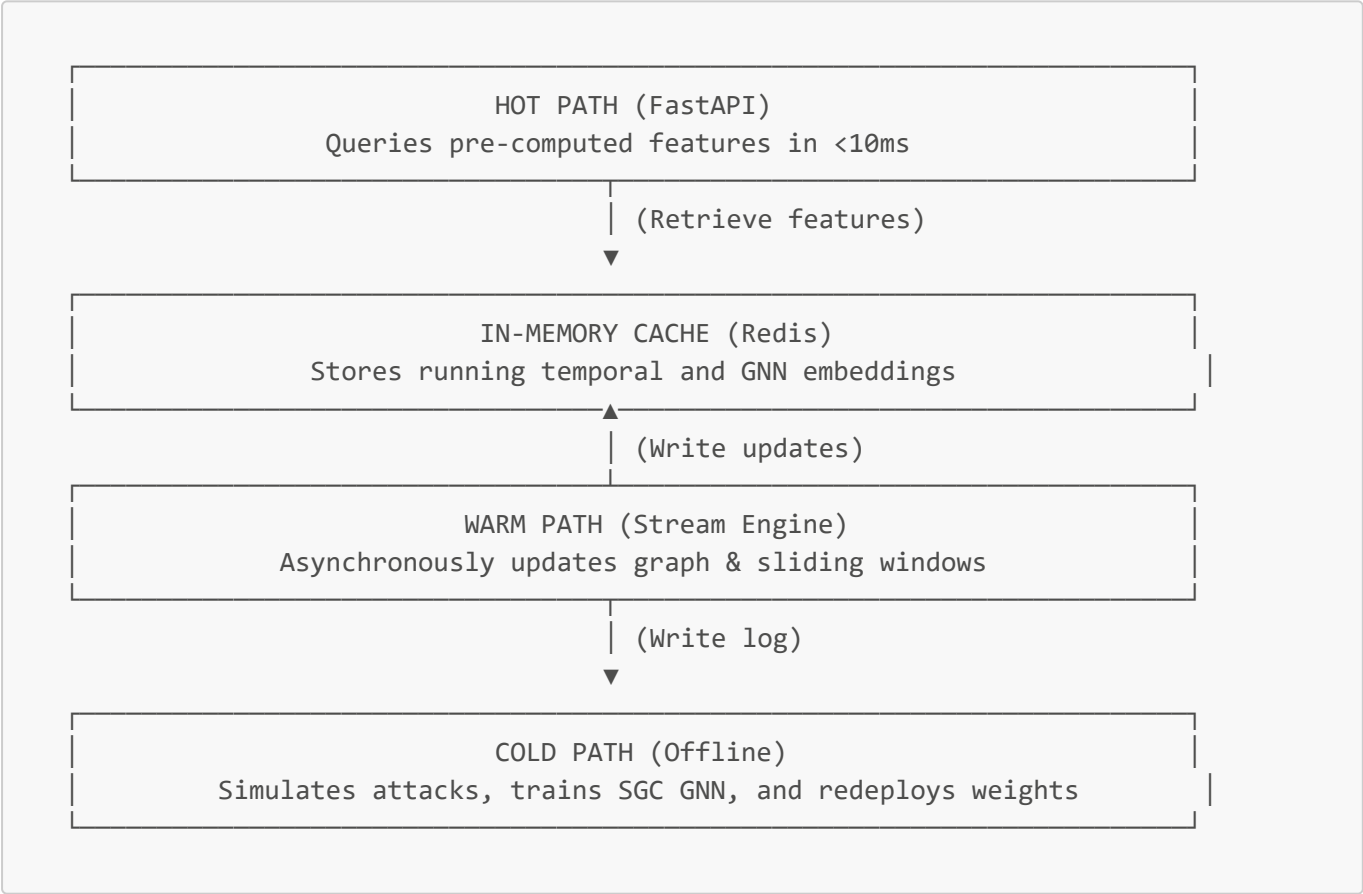
CHAPTER 4: SYSTEM DESIGN AND IMPLEMENTATION

4.1 Architectural Design Philosophy

Modern commercial banking systems (such as those in Nigerian banks like Zenith Bank) process thousands of transactions per second. Real-time transaction switches impose a hard time-to-decision SLA limit (typically < 50ms). Deep graph convolutional neural network (GNN) inference requires significant matrix computation and cannot run synchronously during transaction processing without introducing latency spikes.

To resolve this latency-complexity bottleneck, **Fraudstruct** adopts a **Three-Tier Lambda-Style Architecture**. The philosophy is to **decouple graph updates and training from hot-path lookup**. This ensures that complex structural representations (such as neighborhood anomalies) can be utilized in the synchronous path in $O(1)$ lookup time.

4.2 The Three-Tier System Components



1. Hot Path (Real-Time Inference Layer)

- **Technology:** Python FastAPI + Vectorized Rules Engine.
- **Function:** Runs inside the bank's transaction authorization path. When a customer executes a transfer, the Core Banking System (CBS) queries this API.
- **Mechanism:** The API performs a dictionary lookup to retrieve the sender's pre-computed topological embeddings and running temporal features. It evaluates:

Rule Check = (GNN Anomaly > 0.8) $\vee$ (Rolling Sum > Limit) $\vee$ (Velocity > Threshold)

- **Performance:** Vectorized operations and memory caching restrict execution time to < 5ms.

2. Warm Path (Near Real-Time Feature Ingestion Layer)

- **Technology:** Python Async Event Loop / Bytewax Stream Engine.
- **Function:** Updates the graph topology and temporal features asynchronously after transactions are processed.
- **Mechanism:** Consumes events from a message queue (Kafka/RabbitMQ). For each event, it:
 1. Updates the directed edges in the **GraphEngine** to maintain a real-time transaction network.
 2. Pushes the transaction to a rolling temporal window buffer (**deque**) for the source account.
 3. Re-calculates local features (degree, volume, PageRank).
 4. Writes the updated features immediately to the Redis cache.

3. Cold Path (Offline Training & Validation Layer)

- **Technology:** NumPy / PyTorch GNN Pipeline.
- **Function:** Trains and optimizes the GNN classification models without impacting production workloads.
- **Mechanism:** Periodically (e.g., daily) extracts the transaction graph:
 1. Runs the Graph Adversarial Simulator to inject synthetic smurfing patterns.
 2. Executes SGC convolutions to propagate neighborhood features.
 3. Trains the classifier on validated labels from back-office fraud investigators.
 4. Hot-deploys the newly optimized weights (w and b) directly to the API server via the **/v1/train** endpoint.

4.3 API Schema and Interface Specifications

The API exposes standard REST interfaces for integration with banking networks:

1. Transaction Evaluation Endpoint

Exposes a payload structure compatible with NIBSS Instant Payments (NIP) messages.

- **URL:** **/v1/evaluate**
- **Method:** **POST**
- **Request Header:** **Content-Type:** **application/json**
- **Request Schema:**

```
{
  "type": "object",
  "properties": {
    "transaction_id": { "type": "string" },
    "source_account": { "type": "string" },
    "destination_account": { "type": "string" },
    "amount": { "type": "number" },
    "timestamp": { "type": "string", "format": "date-time" },
    "channel": { "type": "string", "enum": ["NIP", "POS", "ATM", "WEB"] }
  },
}
```

```
"required": ["transaction_id", "source_account", "destination_account",
"amount"]
}
```

- **Response Schema:**

```
{
  "type": "object",
  "properties": {
    "transaction_id": { "type": "string" },
    "decision": { "type": "string", "enum": ["APPROVE", "FLAG", "BLOCK"] },
    "reasons": { "type": "array", "items": { "type": "string" } },
    "latency_ms": { "type": "number" },
    "features": {
      "type": "object",
      "properties": {
        "rolling_sum": { "type": "number" },
        "rolling_count": { "type": "integer" },
        "in_degree": { "type": "integer" },
        "out_degree": { "type": "integer" },
        "gnn_anomaly_score": { "type": "number" }
      }
    }
  }
}
```

2. GNN Retraining Endpoint

Triggered by the Cold Path runner to update active models.

- **URL:** `/v1/train`
- **Method:** `POST`
- **Request Schema:**

```
{
  "type": "object",
  "properties": {
    "labels": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "account_id": { "type": "string" },
          "is_fraud": { "type": "integer", "enum": [0, 1] }
        },
        "required": ["account_id", "is_fraud"]
      }
    },
    "epochs": { "type": "integer", "default": 20 }
  }
}
```

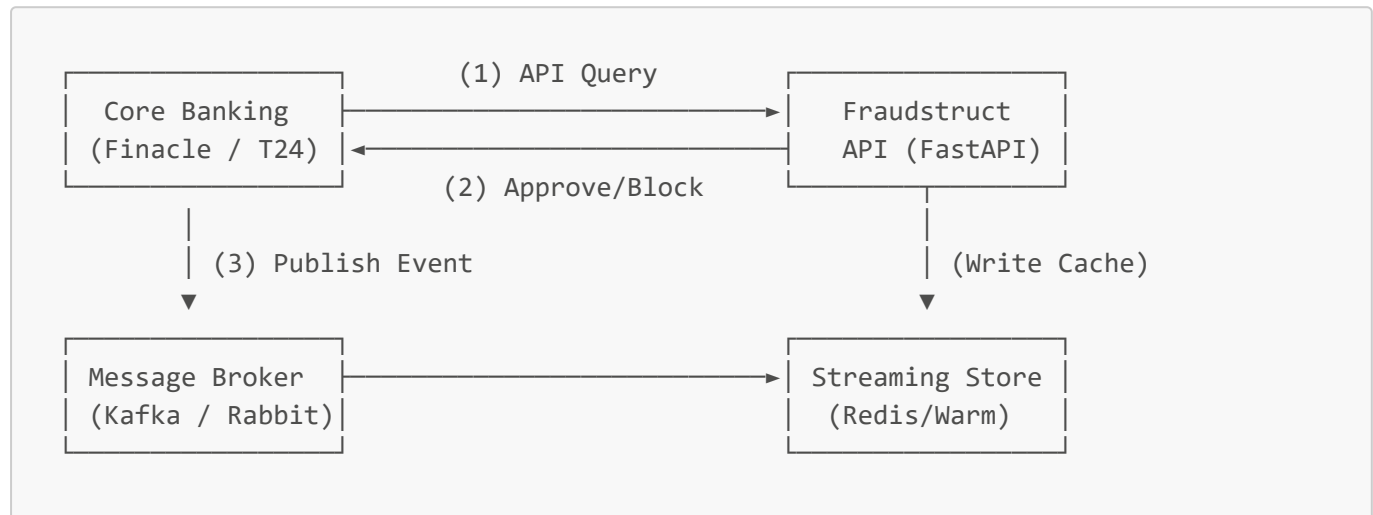
```

    },
    "required": ["labels"]
  }

```

4.4 Core Banking & Inter-Bank Switch Integration

Integrating Fraudstruct into Nigerian banks requires connecting with both internal core infrastructure and central clearing systems:



1. In-Flight Authorization Integration (Finacle/T24)

When a customer initiates a transfer (via mobile app, USSD, or internet banking), the Core Banking System (e.g., Infosys Finacle used by Zenith Bank) executes a synchronous REST API call to `/v1/evaluate`:

- **Outcome APPROVE:** Transaction is authorized.
- **Outcome FLAG:** Transaction is authorized, but a post-authorization ticket is raised for manual compliance review.
- **Outcome BLOCK:** Transaction is aborted, and a security alert is triggered on the account.

2. Inter-Bank NIBSS Instant Payments (NIP) Switch

For inter-bank transfers cleared through the Nigeria Central Switch (NIBSS):

1. The sending bank (Bank A) intercepts the NIP payload.
2. Bank A queries Fraudstruct API.
3. If the GNN node anomaly score shows the destination account at Bank B behaves like a mule, Bank A aborts the transfer before dispatching the NIP message to NIBSS. This prevents outgoing funds leakage.

3. Card Switch Integration (Postilion / ISO 8583)

For card-based transactions processed through switches like Postilion:

1. The switch receives an incoming card ISO 8583 message.
2. A switch middleware parses the card account details and maps them to a REST payload.
3. It queries the `/v1/evaluate` endpoint.
4. The switch denies authorization (returning ISO response code `34` or `57`) if Fraudstruct returns **BLOCK**.

4.5 Non-Functional System Properties

1. High Availability and Fail-Safe Fallbacks

To prevent transaction timeouts if Fraudstruct becomes unavailable:

- **Circuit Breakers:** If the REST API does not respond within 10ms, the core banking hook trips, bypassing GNN validation and falling back to local database-level checks.
- **Heuristic Failover:** If the GNN model weights fail to load, the API automatically falls back to evaluated rolling window threshold checks (`rolling_sum` and `rolling_count`).

2. Memory Optimization and Eviction

To prevent in-memory dequeues from consuming excessive RAM in the Warm Path:

- **Time-to-Live (TTL):** Redis keys are set with a TTL of 3600 seconds (1 hour). Stale account features with zero transaction activity are automatically evicted from RAM.
- **Deque Constraints:** Account queues are capped at a maximum of 100 entries to prevent memory allocation spikes under distributed denial-of-service (DDoS) style cashout attacks.